

SCHOOL OF ENGINEERING

DEPARTMENT OF ELECTRICAL ENGINEERING

BACHELOR OF ELECTRONICS AND COMPUTER ENGINEERING

COURSE TITLE : REAL-TIME SYSTEMS

COURSE CODE : ELE-RTS-521

SUBMITTED TO : M. C. BANDA

SUBMITTED BY : DOREEN PROVIDENCE ABEL

REG NUMBER : BECE/21/SS/001

TOPIC : SOFTWARE ARCHITECTURES

DUE DATE : 3rd November, 2026.

Abstract

This report presents the design and implementation of a responsive microwave oven simulator written in HTML, CSS and JavaScript. The simulator models a simplified microwave oven with a finite state machine (FSM) based on the state diagrams of Assignment 1. The FSM governs transitions between the states waiting, enabled, disabled, cooking, done and alarm. Safety rules prevent the oven from operating with the door open or without a cooking time. The graphical interface is modelled on a real countertop microwave: the door swings open, the interior lamp switches on, and a dim amber light and a turning plate show that cooking is in progress. The countdown uses an end timestamp so that it does not drift, and the whole oven scales with the window size. The program is separated into a model, a view and a controller, and is split across three files (index.html, css/style.css and js/script.js).

1. Introduction

A microwave oven is a small embedded system that moves through a set of well-defined states in response to user inputs such as power selection, time entry and door operation. In real-time systems the correctness of state transitions and the timing of the response are critical (Burns & Wellings, 2009) . A microwave is a convenient example because it must respond promptly to the user, count time accurately and obey a safety rule at all times: it must never operate with the door open.

This project simulates that behaviour in a web browser. It allows a user to set the power level and cooking time, start and stop cooking, open and close the door, and observe the resulting state changes on screen. The design emphasises clarity and separation of concerns so that the logic can be tested independently of the interface.

2. Objectives

1. To implement a microwave simulator with deterministic state transitions that follow the Assignment 1 state diagrams.
2. To design a realistic graphical interface that clearly shows the cooking, standing-by, door-open and door-closed conditions.
3. To make the interface resize sensibly with the window, including on narrow screens.
4. To integrate safety logic that prevents unsafe operation, such as starting with the door open.

5. To keep the code modular, with the logic, layout and styling in separate files.

3. System Requirements

The simulator runs in any recent web browser (for example Chrome, Edge or Firefox). No installation, server or external library is needed. The interface uses CSS container units (MDN Contributors, n.d.-a) and the Web Audio API (MDN Contributors, n.d.-c) , which recent browsers support. The program consists of the files listed in Table 1.

Table 1. Project files

File	Purpose
index.html	Page structure: oven body, door, interior, display and keys
css/style.css	Colour palette, layout, door and light effects, responsive rules
js/script.js	State machine (model), screen update (view) and event handling (controller)
docs/	This report and the screenshots
README.md	Instructions for running and using the simulator

To run the simulator:

1. Keep the folder structure unchanged.
2. Open index.html in a web browser.
3. Click anywhere on the page once. Browsers block sound until the user interacts with the page, and the click allows the buzzer to be heard.

4. System Architecture

The program is divided into four parts:

- **Model (state machine):** the object M holds the current state, the door position, the power level and the remaining time. The object E contains one function for each event (power, add, digit, clear, start, stop and door). The model contains no interface code.
- **View:** the function render() is the only place where the screen is updated. It reads the model and updates the display, the door, the interior light and the status line.

- **Controller:** button handlers call the event functions, and the function tick() runs every 100 ms to advance the countdown and the timed states.
- **Safety logic:** the checks inside E.start and E.door that block starting with the door open, starting with no time set, and running with a fault.

Separating the model from the view means the screen can never disagree with the actual state, because it is always redrawn from the model.

4.1 Timing

A countdown that subtracts one second each time a timer fires drifts, because timer events are never exactly periodic. The simulator therefore stores an end time when cooking starts (end = now + remaining) and, on every tick, computes remaining = end - now using the high-resolution clock performance.now()(MDN Contributors, n.d.-c) . The display stays accurate even if a tick is late. When cooking is stopped or the door is opened, the remaining time is stored so that cooking can resume. The 5-second buzzer and the 3-second alarm use deadlines in the same way.

5. FSM Design

The state machine has six states, described in Table 2. The half-power, full-power and set-time states of assignment 1 are represented by the variables power and remaining time rather than by separate states, which keeps the state set small. The Operation superstate is represented by the states COOK, DONE and ALARM, and the Checking step is performed when Start is pressed.

Table 2. States

State	Meaning	Display	Interior light
WAITING	Idle, door closed, no time set	Current time (clock)	Off
ENABLED	Time set and door closed; Start is allowed	Ready to cook	Off
DISABLED	Door open (or after an alarm); the oven cannot run	Not ready	On
COOK	Cooking; the time counts down	Countdown, power level	On (dim amber)

State	Meaning	Display	Interior light
DONE	Cooking finished; buzzer sounds for 5 seconds	Cooking complete	On
ALARM	Fault found when Start is pressed	Fault message	On

The transitions are listed in Table 3 and drawn in Figure 1.

Table 3. Transitions

From	Event	To	Notes
WAITING	Time set, door closed	ENABLED	Time set by keys or keypad
ENABLED	Start, no fault	COOK	End time stored
ENABLED	Start, fault found	ALARM	Fault simulation
ENABLED	Clear Time or Stop	WAITING	Time cleared
COOK	Time reaches zero	DONE	Buzzer for 5 s
COOK	Stop	ENABLED	Remaining time kept
COOK	Door opened	DISABLED	Cooking stops, time kept
DONE	5 s ends, or Stop	WAITING	Time is zero
ALARM	3 s ends	DISABLED	Door must be cycled
Any other state	Door opened	DISABLED	Light on
DISABLED	Door closed	ENABLED or WAITING	ENABLED if time is set

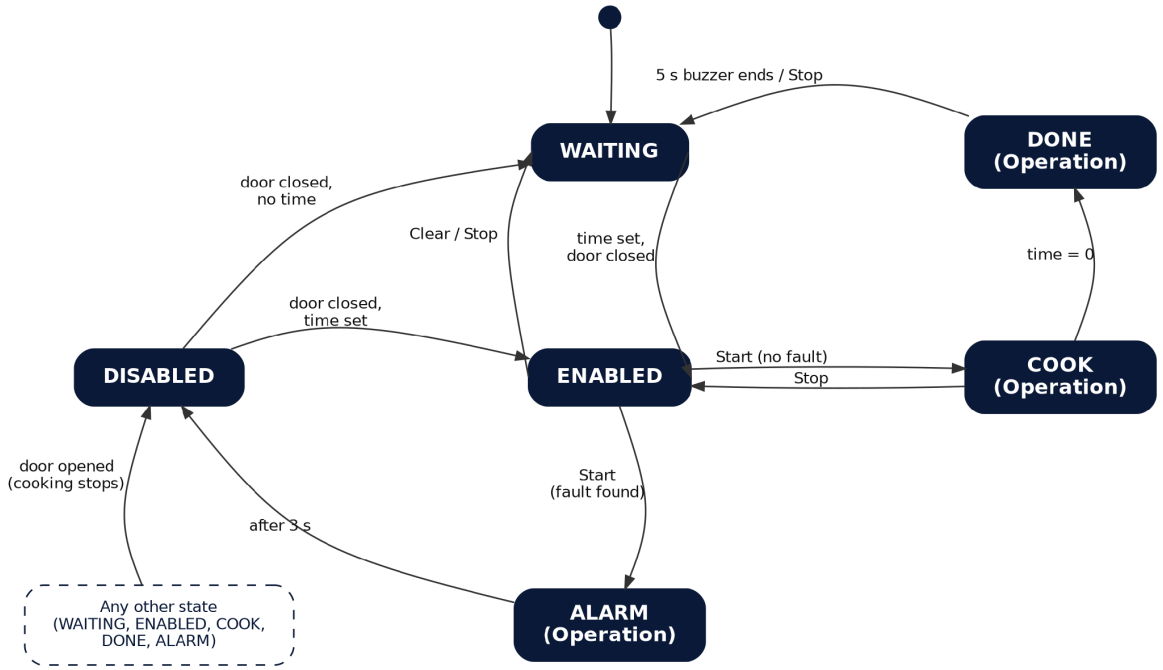


Figure 1. State diagram of the microwave oven simulator

6. User Interface Design

The interface is modelled on a real countertop microwave oven, with proportions of about 1.75 to 1. It has two main areas:

- **Door and food container (left):** a large door with a smoked-glass window. Behind the glass is the oven interior with a glass plate and food.
- **Control panel (right):** a blue display showing the time and a message line, the power keys (Half Power 300 W and Full Power 600 W), the time keys (+10s, +1m, +10m and Clear Time), a numeric keypad, and the Start, Stop and Open/Close Door keys. A status line below the keys shows the state, door position and power level.

The key layout follows the group layout of the sample interface. Table 4 shows how each condition of the food container is displayed.

Table 4. Visual states of the food container

Condition	How it is shown
Door closed, standing by	Dark smoked glass with a mesh of small dots; the interior is dark
Door open	The door swings out on its hinge; the interior lamp is on and the plate and food are visible

Condition	How it is shown
Cooking	Dim amber light, brightest at the centre of the window; the plate turns and the dots show against the glow
Done	The lamp stays on while the buzzer sounds for 5 seconds



Figure 2. Door closed, standing by

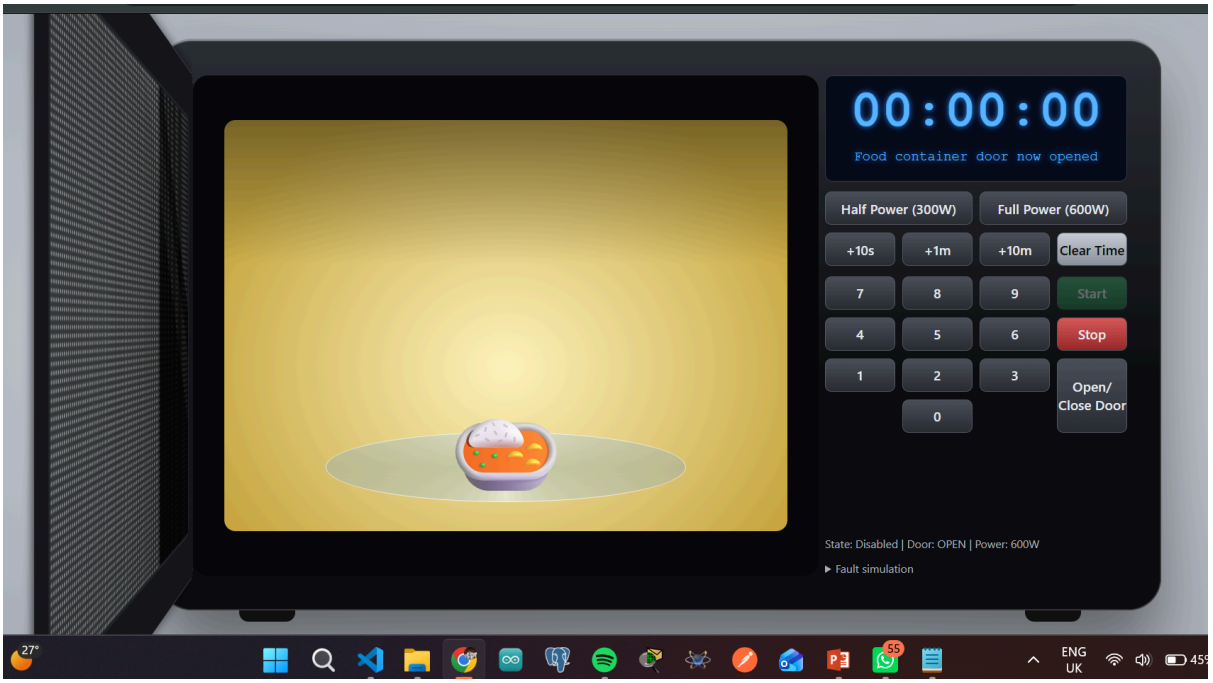


Figure 3. Door open



Figure 4. Cooking

6.1 Responsive layout

The oven is sized with CSS container units, so every part (door, display, keys and text) scales together and keeps the proportions of a real oven as the window changes. Text and key sizes are stored as variables and have minimum sizes so that they stay readable. Below a width of 700 px the layout stacks vertically, with the door above the controls and touch-sized keys. Further rules cover tablets, very small phones and short landscape windows, and the animations are switched off for users who prefer reduced motion.



Figure 5. Layout on a narrow window

7. Operational Flow

A typical cooking sequence is:

1. The user selects a power level (Half or Full Power). Full Power is the default.
2. The user sets the cooking time with the time keys or the keypad. The oven moves to ENABLED if the door is closed.
3. The user presses Start. With the door closed, time set and no fault, the oven moves to COOK.
4. The display counts down in hh:mm:ss format, the interior light is on, and the plate turns.

5. At zero the oven moves to DONE, the buzzer sounds for 5 seconds, and the screen shows "Cooking complete". The oven then returns to WAITING.
6. If the door is opened during cooking, cooking stops, the remaining time is kept, and the screen shows "Food container door now opened".

Pressing Stop while cooking stops the oven and keeps the time; pressing Stop a second time clears the time.

8. Implementation Details

- **Time entry:** the +10s, +1m and +10m keys add to the current time, so a key can be pressed several times. The keypad works like a real oven: digits fill minutes and seconds from the right, so pressing 1, 2, 3, 0 sets 12:30. The maximum time is 99:59:59.
- **Messages:** the screen shows "Food container door now opened/closed" when the door button is pressed, "Door is open. Cannot start." and "Set cooking time first" when Start is refused, and "Cooking complete" at the end.
- **Buzzer:** a square-wave tone at 880 Hz is generated with the Web Audio API (MDN Contributors, n.d.-c) and switched on and off every 250 ms for 5 seconds.
- **Fault simulation:** two checkboxes (turntable fault and emitter fault) allow the Checking step to be demonstrated. With a fault selected, pressing Start shows an alarm message and moves the oven to DISABLED.
- **Colours:** every colour is defined once, as a CSS variable at the top of style.css, and the rest of the file refers to the variables.
- **Files:** the structure, styling and logic are kept in separate files, and the logic is divided into the model, view and controller described in Section 4.

9. Testing and Evaluation

Testing is carried out in three phases:

- **Functional testing:** checks that each key produces the correct state transition and display.
- **Safety testing:** checks that unsafe operations, such as starting with the door open, are blocked.

- **Visual testing:** checks that the layout and text adjust correctly when the window is resized.

The test cases are listed in Table 5.

Table 5. Test cases

#	Test	Expected result	Result
1	Press +10s three times	Display shows 00:00:30	pass
2	Press 1, 2, 3, 0 on the keypad	Display shows 00:12:30	pass
3	Press Clear Time	Display shows 00:00:00	pass
4	Set 10 s and press Start	Countdown runs, light on, plate turns	pass
5	Let the countdown reach zero	Buzzer for 5 s, "Cooking complete", then standby	
6	Press Stop once while cooking	Cooking stops, time kept	pass
7	Press Stop a second time	Time cleared	pass
8	Open the door while cooking	Cooking stops, "Food container door now opened"	pass
9	Press Start with the door open	Not started, "Door is open. Cannot start."	pass
10	Press Start with no time set	Not started, "Set cooking time first"	pass
11	Tick a fault, set a time, press Start	Alarm message, then "Not ready"	pass
12	Resize the window while cooking	Layout adapts and the countdown continues	pass

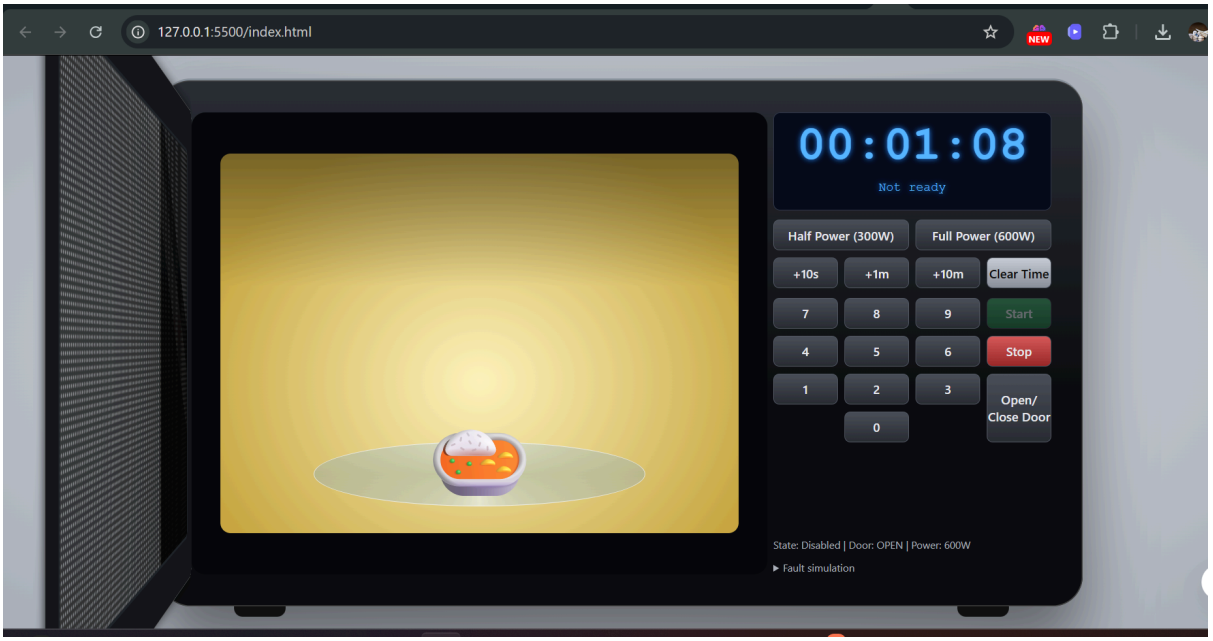


Figure 6. Starting with the door open is blocked

10. Future Work

Possible improvements include:

- Different power levels that change the cooking effect, with the 300 W and 600 W levels affecting the cooking time.
- Preset cooking programs and a defrost mode.
- A rotary dial and a real-photo door image for greater realism.
- Distinct audio alerts for the end of cooking and for faults.
- Real-time logging of state changes to help with testing.

11. Conclusion

The simulator implements the state machine of Assignment 1 in a realistic, resizable interface. Separating the model, view and controller keeps the logic simple to test, and storing an end timestamp keeps the countdown accurate. The safety rules prevent the oven from running with the door open or without a cooking time.

References

1. Malawi University of Business and Applied Sciences. (2026). RTS 521 Real-Time Systems, Assignment 2
3. Burns, A., & Wellings, A. J. (2009). Real-time systems and programming languages (4th ed.). Addison-Wesley.
4. MDN Web Docs. Web Audio API. https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API
5. MDN Web Docs. CSS container queries. https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_containment/Container_queries
6. MDN Web Docs. Performance: now() method. https://developer.mozilla.org/en-US/docs/Web/API/Performance/now